

AI AUTOMATION PROJECT

Implementation, Testing & Evidence Report

Python | Supabase | Ollama | GitHub | Windows Task Scheduler

Evidence period: October 2026

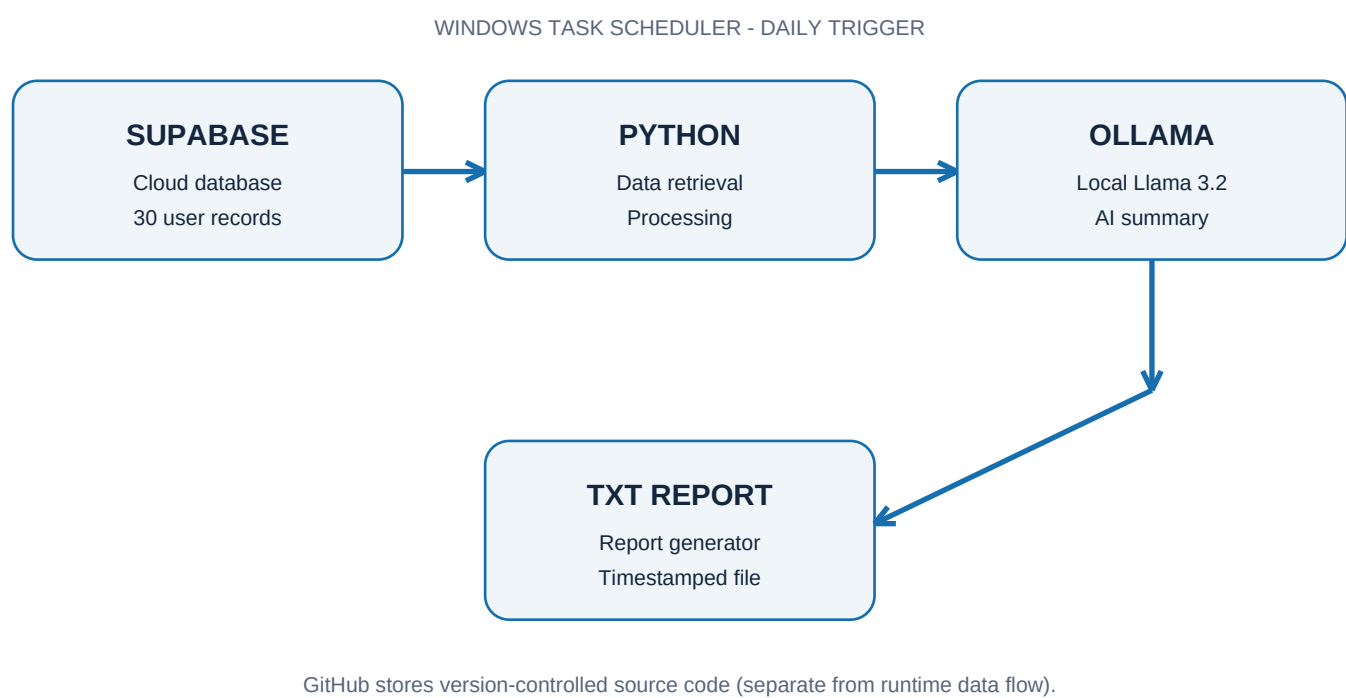
Document type: Final Year Project - Technical Evidence

Prepared from available project screenshots and observed test outputs.

1. Project Overview

The project demonstrates an AI-assisted reporting workflow. A Python application retrieves records from a Supabase table, prepares user statistics, requests a natural-language summary from a locally running Ollama model, and writes a timestamped text report. GitHub provides source version control. Windows Task Scheduler launches the reporting command on a daily schedule.

2. System Architecture



Architecture notes

Data source: Supabase public.test_users (30 test records). **Application:** Python modules under src/. **AI engine:** Ollama using a Llama 3.2 model. **Output:** reports/user_report_YYYYMMDD_HHMMSS.txt. **Automation:** Windows Task Scheduler runs the Python reporting module.

3. Verified Outcomes

Verification	Observed evidence
Supabase API connectivity	Successful connection; accessible rows reported
Local AI integration	Ollama returned an AI response
AI + database integration	Five records fetched and summarized in integration test
Report generation	30-user TXT report saved successfully

Verification	Observed evidence
Scheduled-task manual execution	Last Run Result: 0x0 (success)
GitHub version control	Push to main completed; README visible online

4. Implementation Evidence

Evidence 01 - Python source code and Ollama integration

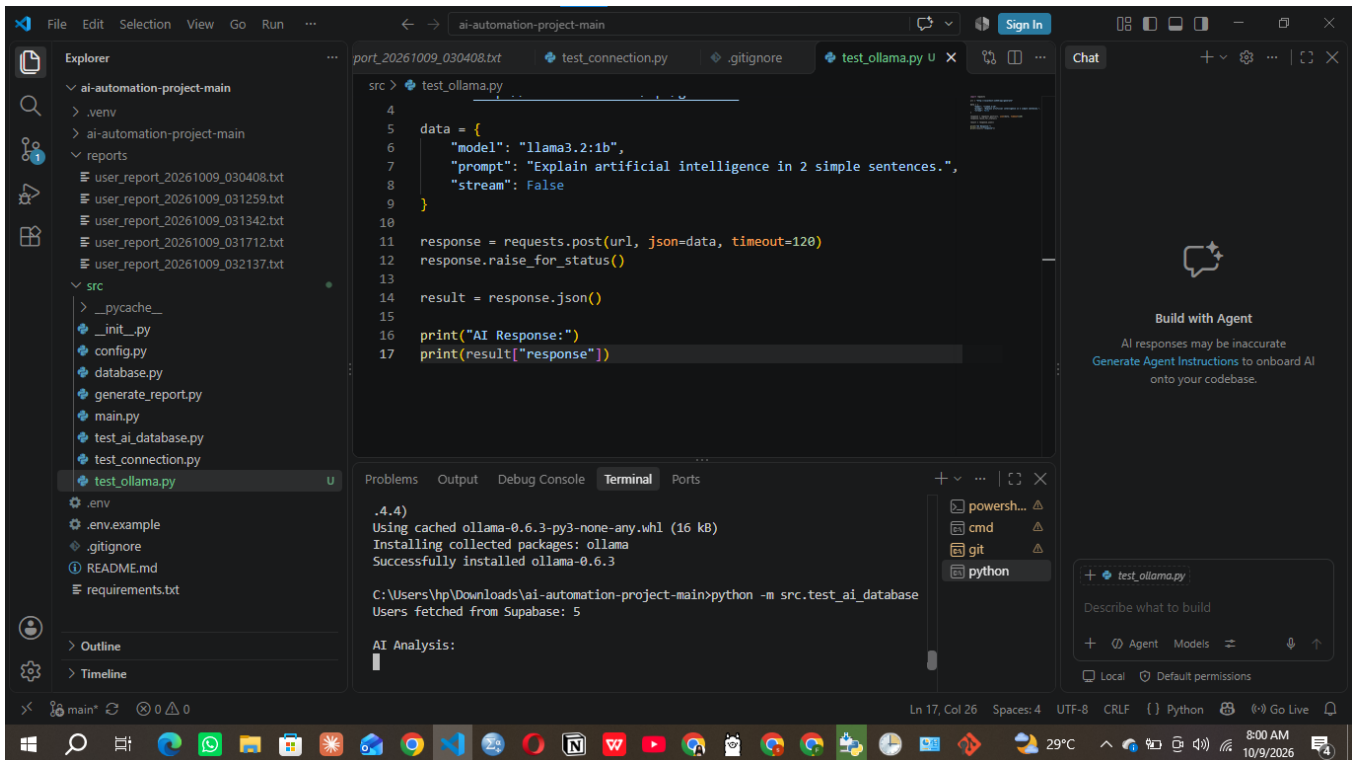


Figure 1. Python Ollama request and successful AI response visible in VS Code.

Evidence 02 - AI/database integration

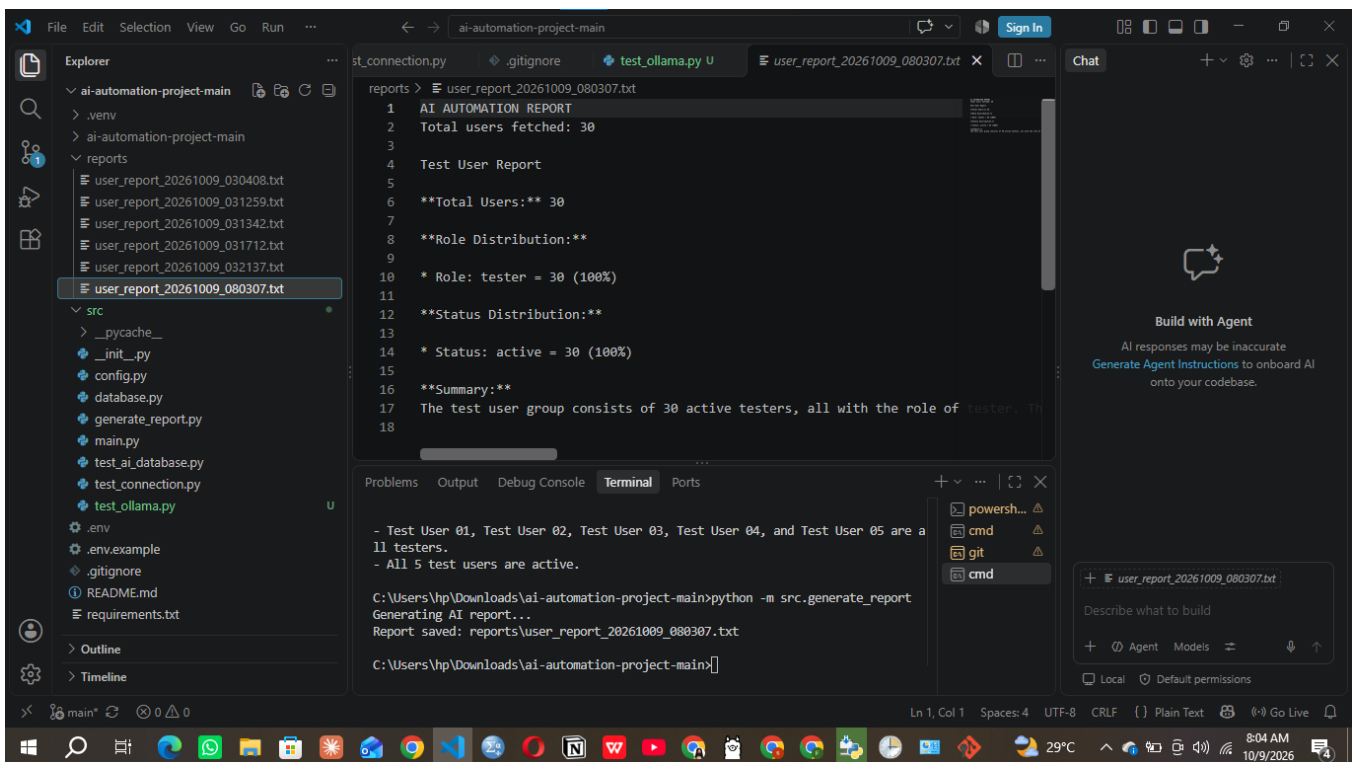
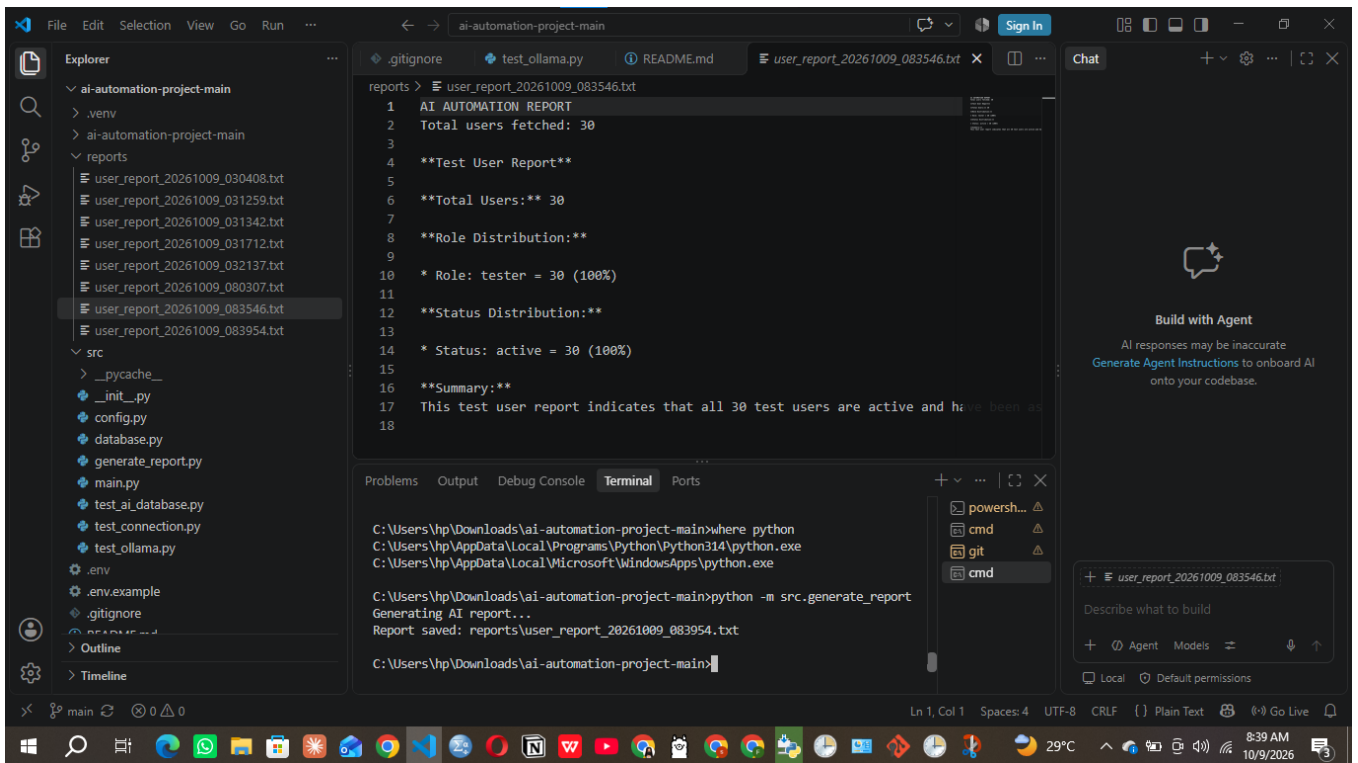


Figure 2. Integration testing and user summary in the local development environment.

5. Report Generation Evidence

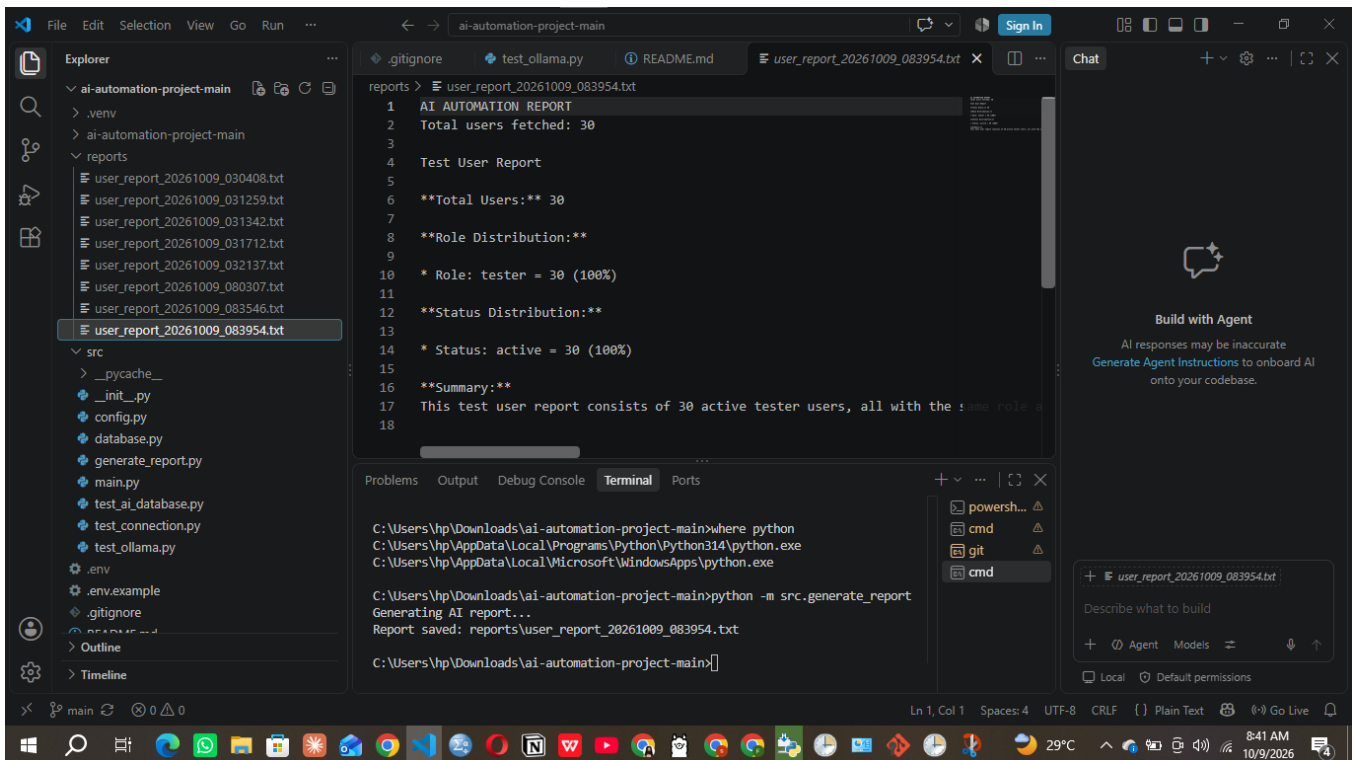


The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows a project named 'ai-automation-project-main' with a 'reports' folder containing several timestamped TXT files. The editor displays the content of 'user_report_20261009_083546.txt', which contains the following text:

```
1 AI AUTOMATION REPORT
2 Total users fetched: 30
3
4 **Test User Report**
5
6 **Total Users:** 30
7
8 **Role Distribution:**
9
10 * Role: tester = 30 (100%)
11
12 **Status Distribution:**
13
14 * Status: active = 30 (100%)
15
16 **Summary:**
17 This test user report indicates that all 30 test users are active and have been as
18
```

The terminal at the bottom shows the command 'python -m src.generate_report' being executed, with the output 'Generating AI report...' and 'Report saved: reports/user_report_20261009_083954.txt'.

Figure 3. Successful execution of `python -m src.generate_report`; timestamped TXT report created.



The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows the same project structure as Figure 3, but the 'reports' folder now contains a file named 'user_report_20261009_083954.txt'. The editor displays the content of this file, which contains the following text:

```
1 AI AUTOMATION REPORT
2 Total users fetched: 30
3
4 Test User Report
5
6 **Total Users:** 30
7
8 **Role Distribution:**
9
10 * Role: tester = 30 (100%)
11
12 **Status Distribution:**
13
14 * Status: active = 30 (100%)
15
16 **Summary:**
17 This test user report consists of 30 active tester users, all with the same role a
18
```

The terminal at the bottom shows the same command and output as in Figure 3.

Figure 4. Generated report showing 30 users, role distribution, status distribution and AI summary.

6. Automation & Version Control Evidence

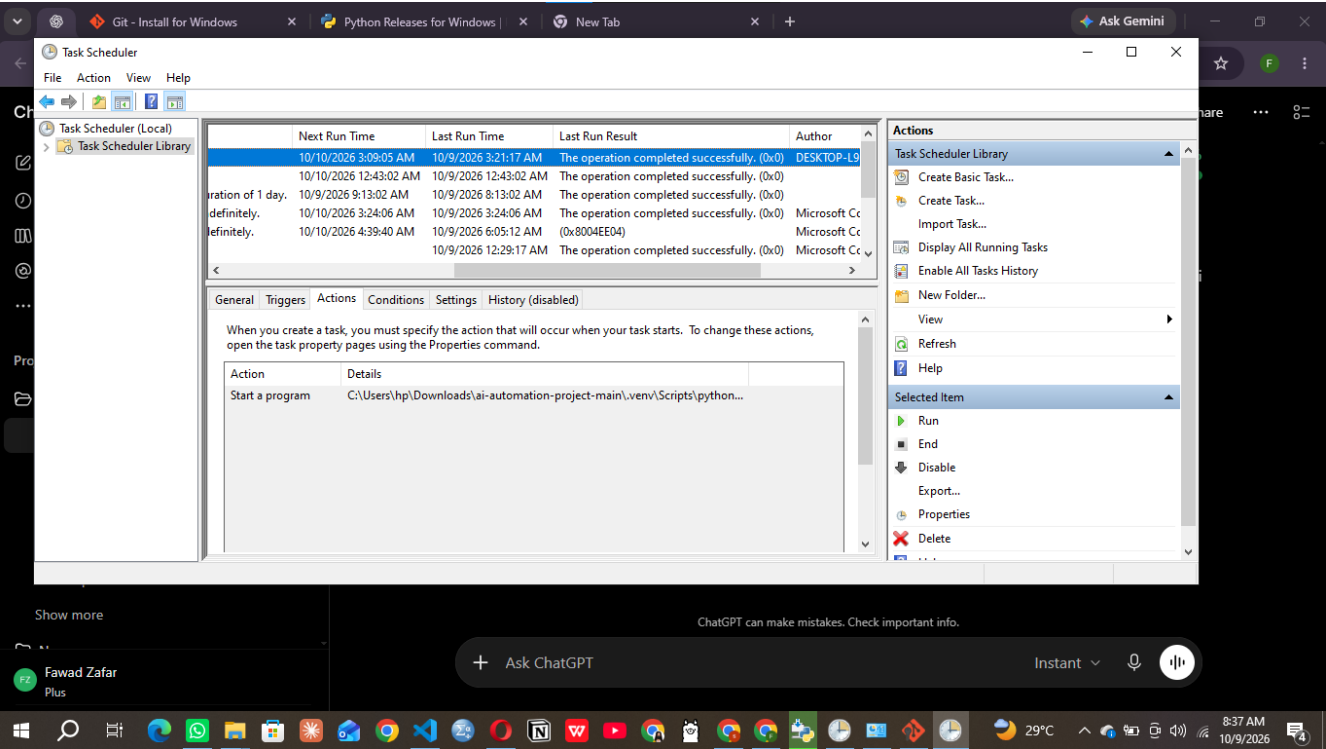


Figure 5. Task Scheduler: daily task and Last Run Result showing successful completion (0x0).

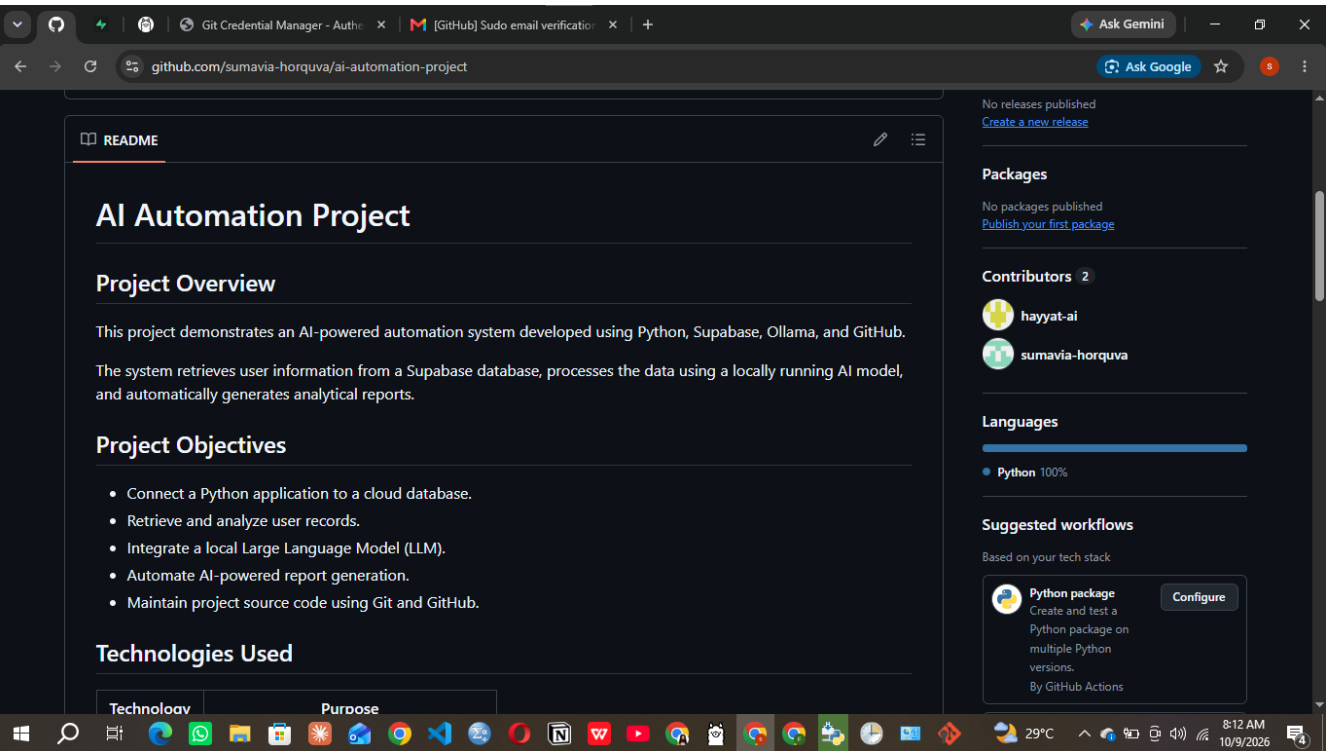


Figure 6. GitHub repository README, project objectives and technology overview.

7. Database Configuration Evidence

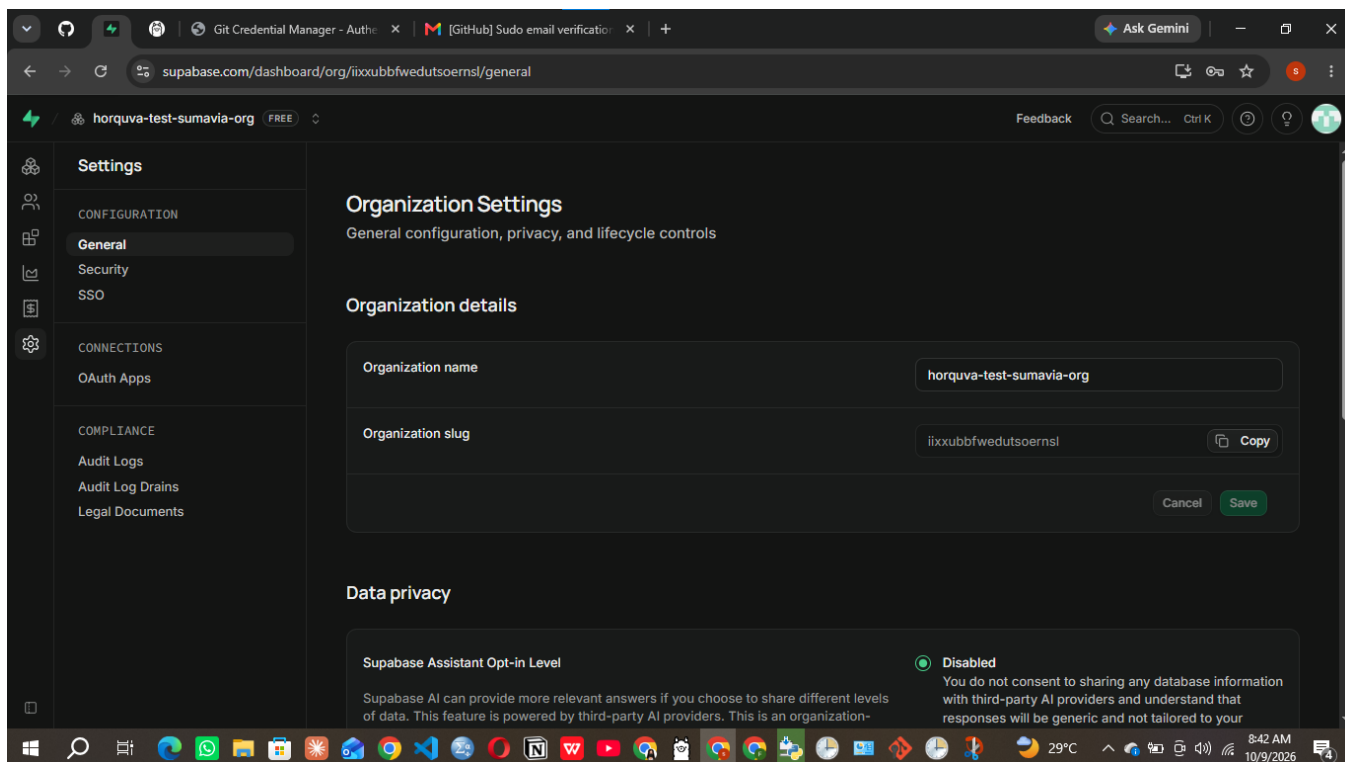


Figure 7. Supabase organization settings screen. This confirms dashboard access, not table contents.

Database verification statement

The successful Python report contains 30 records, all marked active and assigned the tester role. A separate screenshot of the Supabase Table Editor showing the test_users table was not available in the selected evidence. The database record count is therefore supported by application output rather than a direct table screenshot.

8. Remaining Verification & Limitations

Unattended daily run: A manual launch from Windows Task Scheduler completed successfully. The next unattended scheduled run has not been evidenced here; verify that it generates a new report at the configured time.

Runtime requirements: The computer must be available, the configured Windows account must meet task conditions, the Python environment must be accessible, and Ollama must be running.

Scope: This evidence supports a working prototype and test dataset. It does not establish production readiness, live-user analytics, or independent verification of every scheduled run.

9. Conclusion

The observed results demonstrate a functioning end-to-end AI reporting prototype: Supabase-backed user statistics, local language-model summarization, timestamped report export, GitHub source tracking and a successful manually triggered scheduled execution. Final unattended-run evidence and a direct database-table screenshot would strengthen the submission.